

Rapport de réalisation

Etat d'avancement :

Réalisation de *la classification binaire*
(*BENIGN/ATTACK*)

Encadre par :

M. Azizi Mohamed Saad (ENSIASD)
Saad Nasihi (SOREMED)

Réalisé par :

Ait El Hadj Mohamed

Année universitaire : 2026-2027

Table des matières

<i>Rapport de réalisation</i>	1
1. Introduction	4
2. Mise en œuvre de la classification binaire	4
2.1 Transformation des étiquettes (BENIGN / ATTACK)	4
2.2 Séparation des données d'entraînement et de test	4
2.3 Métriques d'évaluation retenues	5
3. Développement des modèles machine Learning	6
3.1 Arbre de Décision (Decision Tree)	6
3.2 Forêt Aléatoire (Random Forest)	6
3.3 XGBoost	6
4. Comparaison des performances des modèles	6
4.1 Comparaison des métriques de performance	6
4.2 Analyse de l'influence de la variable Destination Port	6
4.3 Sélection du modèle le plus performant	6
5. Développement de l'application de démonstration	6
5.1 Architecture de l'application Streamlit	6
5.2 État d'avancement de l'interface utilisateur	6
6. Synthèse de l'état d'avancement	6

1. Introduction

2. Mise en œuvre de la classification binaire

2.1 Transformation des étiquettes (BENIGN / ATTACK)

Le jeu de données CICIDS2017 contient initialement quinze catégories correspondant à différents types d'attaques ainsi qu'une classe représentant le trafic normal (**BENIGN**). Dans le cadre de cette première phase de développement, toutes les catégories d'attaques ont été fusionnées afin de constituer une unique classe **ATTACK**.

Cette transformation permet de reformuler le problème sous la forme d'une classification binaire :

- **BENIGN** : trafic réseau normal.
- **ATTACK** : ensemble des activités malveillantes (DoS, DDoS, PortScan, Bot, FTP-Patator, SSH-Patator, Web Attacks, Heartbleed, Infiltration, etc.).

Après cette transformation, la distribution des classes obtenue est la suivante :

Classe	Nombre d'instances
BENIGN	2 095 057
ATTACK	425 741

Cette répartition met en évidence un déséquilibre entre les deux classes, le trafic normal représentant la majorité des observations. Ce déséquilibre a été pris en compte lors de l'évaluation des modèles en privilégiant des métriques adaptées, telles que le rappel (Recall).

2.2 Séparation des données d'entraînement et de test

Après la préparation des données, le jeu de données a été divisé en deux sous-ensembles : un ensemble d'entraînement destiné à l'apprentissage des modèles et un ensemble de test réservé à l'évaluation de leurs performances.

Une séparation **stratifiée** a été utilisée afin de conserver la même proportion des classes **BENIGN** et **ATTACK** dans les deux ensembles. Cette méthode est particulièrement adaptée aux jeux de données déséquilibrés, car elle garantit une représentation équitable des différentes classes lors de l'entraînement et de l'évaluation.

La répartition retenue est de **80 %** des données pour l'entraînement et **20 %** pour le test, avec un paramètre `random_state` fixé afin d'assurer la reproductibilité des expériences.

2.3 Métriques d'évaluation retenues

L'évaluation des modèles ne repose pas uniquement sur le taux de précision global (**Accuracy**), celui-ci pouvant être trompeur dans un contexte où la classe **BENIGN** est largement majoritaire. Un modèle prédisant systématiquement la classe majoritaire pourrait ainsi obtenir une précision élevée sans être réellement capable de détecter les attaques.

Afin d'obtenir une évaluation plus représentative des performances, plusieurs métriques ont été retenues :

- **Accuracy** : proportion globale des prédictions correctes.
- **Precision** : capacité du modèle à limiter les faux positifs.
- **Recall** : capacité du modèle à détecter les attaques réelles, en limitant les faux négatifs.
- **F1-score** : moyenne harmonique entre la précision et le rappel.
- **Macro F1-score** : moyenne des F1-scores calculée indépendamment pour chaque classe, permettant une meilleure prise en compte du déséquilibre du jeu de données.
- **Matrice de confusion** : représentation détaillée des bonnes et mauvaises classifications pour chaque classe.

3. Développement des modèles machine Learning

3.1 Arbre de Décision (Decision Tree)

L'arbre de décision (Decision Tree) est un algorithme d'apprentissage supervisé largement utilisé pour les problèmes de classification et de régression. Son principe repose sur la construction d'une structure arborescente dans laquelle chaque nœud représente une règle de décision appliquée à une variable du jeu de données, tandis que chaque branche correspond au résultat de cette règle. Le processus se poursuit jusqu'à atteindre des feuilles terminales, qui représentent la classe prédite.

Lors de la phase d'apprentissage, l'algorithme sélectionne à chaque étape la variable offrant la meilleure séparation des données selon un critère d'impureté, tel que l'indice de Gini ou l'entropie. L'objectif est de créer des groupes les plus homogènes possibles afin de maximiser la capacité de discrimination entre les différentes classes.

Dans le contexte de ce projet, l'arbre de décision constitue un premier modèle de référence (baseline). Il présente plusieurs avantages, notamment sa simplicité d'interprétation,

sa rapidité d'entraînement ainsi que sa capacité à gérer directement des variables numériques sans nécessiter de normalisation préalable des données. En revanche, un arbre trop profond peut conduire à un phénomène de surapprentissage (overfitting), ce qui nécessite de contrôler certains paramètres de l'algorithme.

La Figure 1 illustre le principe général de fonctionnement d'un arbre de décision.

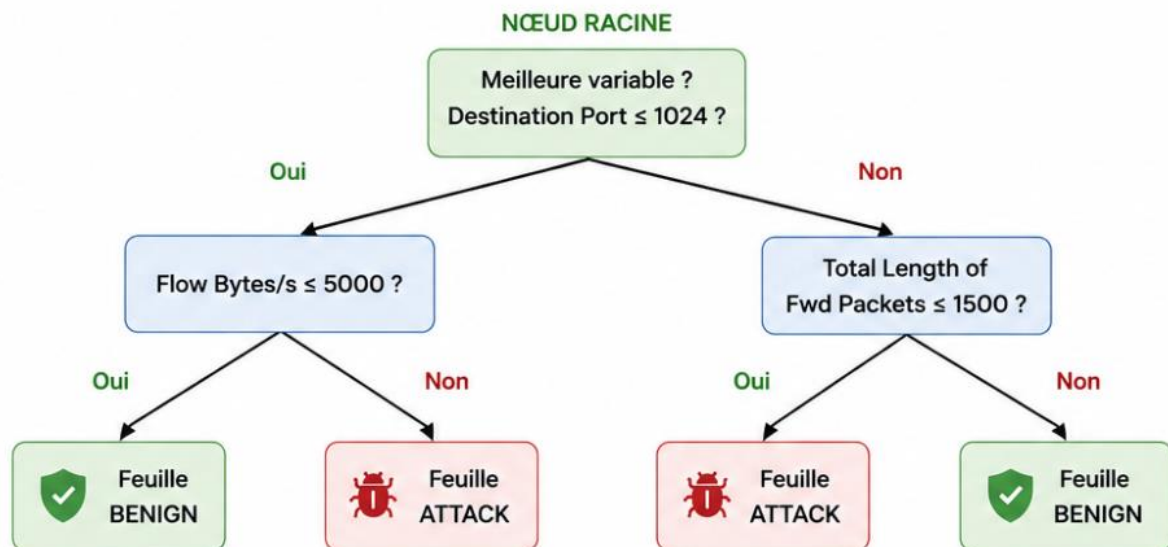


Figure 1: Principe de fonctionnement de l'algorithme Decision Tree

✓ Modèle avec la variable *Destination Port*

Dans une première expérimentation, le modèle a été entraîné en utilisant l'ensemble des variables du jeu de données, y compris la variable **Destination Port**. Cette approche permet d'exploiter toutes les informations disponibles afin d'obtenir les meilleures performances possibles en matière de détection.

La variable **Destination Port** correspond au port réseau de destination utilisé lors de la communication. Dans le jeu de données CICIDS2017, certaines catégories d'attaques ciblent fréquemment des ports spécifiques. Cette caractéristique peut donc constituer un indicateur pertinent pour distinguer un trafic légitime d'un trafic malveillant.

Le modèle a été entraîné sur l'ensemble d'apprentissage puis évalué sur l'ensemble de test à l'aide des métriques retenues : Accuracy, Precision, Recall, F1-score, Macro F1-score ainsi que la matrice de confusion. Le temps d'entraînement a également été mesuré afin de comparer les performances avec les autres algorithmes étudiés.

✓ Modèle sans la variable *Destination Port*

Afin d'évaluer l'influence réelle de la variable **Destination Port**, une seconde expérimentation a été réalisée après sa suppression du jeu de données avant l'entraînement du modèle.

Cette expérimentation répond à une recommandation de l'encadrant et vise à vérifier si les performances du modèle reposent principalement sur cette variable ou si celui-ci est capable

d'apprendre les comportements caractéristiques du trafic réseau à partir des autres attributs disponibles.

Les mêmes paramètres d'entraînement ainsi que la même stratégie de séparation des données ont été conservés afin de garantir une comparaison équitable entre les deux modèles. Les résultats obtenus permettront d'analyser l'impact de la suppression de **Destination Port** sur les performances globales de l'algorithme et d'évaluer sa capacité de généralisation dans un environnement réseau réel.

3.2 Forêt Aléatoire (Random Forest)

La **Forêt Aléatoire (Random Forest)** est un algorithme d'apprentissage supervisé basé sur le principe de l'apprentissage par ensemble (*Ensemble Learning*). Contrairement à l'arbre de décision, qui construit un unique arbre de classification, le Random Forest génère un ensemble de plusieurs arbres de décision indépendants, appelés **arbres faibles** (*weak learners*). La prédiction finale est obtenue par un vote majoritaire des prédictions produites par chaque arbre.

L'objectif de cette approche est de réduire le risque de surapprentissage (*overfitting*) observé avec un arbre de décision unique et d'améliorer la capacité de généralisation du modèle.

Le processus de construction d'une forêt aléatoire repose sur deux principes fondamentaux :

- **Bootstrap Sampling** : chaque arbre est entraîné sur un sous-ensemble aléatoire du jeu d'entraînement obtenu par tirage avec remise.
- **Sélection aléatoire des variables** : à chaque nœud, seule une partie des variables est considérée pour effectuer la meilleure séparation.

Ces deux mécanismes introduisent de la diversité entre les arbres, ce qui améliore la robustesse globale du modèle.

Mathématiquement, si la forêt est composée de (N) arbres de décision, chaque arbre produit une prédiction ($h_i(x)$) pour une observation (x).

La prédiction finale est obtenue par vote majoritaire :

$$\hat{y} = \text{mode}(h_{1(x)}, h_{2(x)}, \dots, h_{n(x)})$$

Où :

- ($h_i(x)$) représente la prédiction du ($i^{\text{ème}}$) arbre ;
- (N) est le nombre total d'arbres de la forêt.

Chaque arbre est construit selon le même principe qu'un arbre de décision classique. Pour sélectionner la meilleure séparation, le Random Forest utilise généralement **l'indice de Gini**, défini par :

$$Gini = 1 - \sum_{i=1}^{\{C\}} p_i^2$$

Où :

- (C) représente le nombre de classes ;
- (p_i) correspond à la proportion des observations appartenant à la classe (i).

Le meilleur découpage est celui qui minimise l'impureté des nœuds.

Comparé à un arbre de décision unique, le Random Forest présente plusieurs avantages :

- Meilleure capacité de généralisation ;
- Réduction du surapprentissage ;
- Bonne robustesse face au bruit ;
- Capacité à traiter des jeux de données de grande dimension.

En revanche, cette amélioration des performances se traduit par un temps d'entraînement plus important ainsi qu'une interprétation moins intuitive du modèle.

La Figure 2 illustre le principe général de fonctionnement de l'algorithme Random Forest.

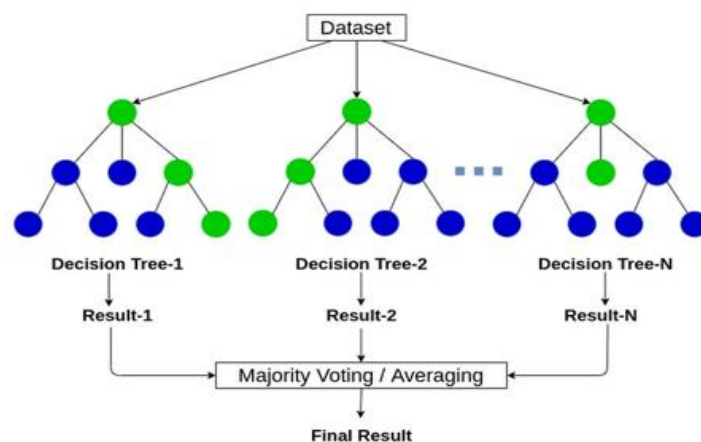


Figure 2: Principe de fonctionnement de l'algorithme Random Forest

✓ **Modèle avec la variable *Destination Port***

Dans une première expérimentation, le modèle Random Forest a été entraîné en utilisant l'ensemble des variables du jeu de données, y compris la variable **Destination Port**.

Chaque arbre composant la forêt reçoit un sous-échantillon aléatoire des données d'entraînement ainsi qu'un sous-ensemble aléatoire des variables disponibles. Les prédictions individuelles sont ensuite combinées par vote majoritaire afin de déterminer la classe finale (**BENIGN** ou **ATTACK**).

Cette première configuration permet d'évaluer les performances maximales du modèle lorsque toutes les caractéristiques réseau sont disponibles.

Les performances ont été évaluées à l'aide des métriques suivantes :

- Accuracy ;
- Precision ;
- Recall ;
- F1-score ;
- Macro F1-score ;
- Matrice de confusion.

Le temps d'entraînement a également été mesuré afin de comparer le coût de calcul avec celui des autres modèles étudiés.

✓ **Modèle sans la variable *Destination Port***

Une seconde expérimentation a été réalisée après suppression de la variable **Destination Port** avant l'entraînement du modèle.

Cette comparaison a pour objectif d'étudier l'influence réelle de cette variable sur les performances du Random Forest. En effet, dans le jeu de données CICIDS2017, certaines attaques utilisent des ports spécifiques, ce qui peut conduire le modèle à apprendre des règles fortement dépendantes de cette caractéristique.

En supprimant cette variable, le modèle est contraint d'exploiter uniquement les autres attributs décrivant le comportement du trafic réseau, tels que les durées de flux, les tailles des paquets, les débits ou les intervalles de temps entre paquets.

Les mêmes paramètres d'entraînement, la même séparation stratifiée des données ainsi que les mêmes métriques d'évaluation ont été conservés afin d'assurer une comparaison objective entre les deux modèles. Les résultats obtenus permettront d'analyser l'impact de la variable **Destination Port** sur la qualité de la détection et sur la capacité de généralisation du modèle Random Forest.

3.3 XGBoost

XGBoost (*eXtreme Gradient Boosting*) est un algorithme d'apprentissage supervisé basé sur la technique du **Gradient Boosting**. Contrairement au Random Forest, qui construit plusieurs arbres de décision de manière indépendante, XGBoost construit les arbres **de façon séquentielle**. Chaque nouvel arbre est entraîné pour corriger les erreurs commises par les arbres précédents, ce qui permet d'améliorer progressivement les performances du modèle.

Le principe de fonctionnement consiste à ajouter successivement des arbres de décision faibles (*weak learners*), chacun étant chargé de réduire les erreurs de prédiction du modèle précédent. La prédiction finale correspond à la somme des contributions de tous les arbres construits.

Mathématiquement, après (K) itérations, la prédiction est donnée par :

$$\hat{y} = f_{1(x)} + f_{2(x)} + \dots + f_{k(x)}$$

où :

- $(\hat{y})$ est la prédiction finale de l'observation (i) ;
- (f_k) représente le $(k^{\text{ème}})$ arbre de décision ;
- (K) est le nombre total d'arbres.

À chaque itération, XGBoost cherche à minimiser une fonction objectif composée de deux parties : l'erreur de prédiction et un terme de régularisation destiné à limiter le surapprentissage.

La fonction objectif est définie par :

$$Obj = \sum_{i=1}^n l(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

Avec

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

où :

- $l(y_i, \hat{y}_i)$ représente la fonction de perte ;
- (T) est le nombre de feuilles de l'arbre ;
- (w_j) est le poids associé à une feuille ;
- (γ) et (λ) sont des paramètres de régularisation.

Cette régularisation constitue l'une des principales différences entre XGBoost et les méthodes classiques de Gradient Boosting, en améliorant la capacité de généralisation du modèle.

Par rapport aux autres algorithmes étudiés, XGBoost présente plusieurs avantages :

- Excellente précision de classification ;
- Réduction du surapprentissage grâce à la régularisation ;
- Gestion efficace des jeux de données de grande taille ;
- Optimisation des performances par calcul parallèle.

En contrepartie, son entraînement est généralement plus long qu'un arbre de décision simple et nécessite davantage de paramètres de configuration.

La Figure 3 présente le principe général de fonctionnement de l'algorithme XGBoost.

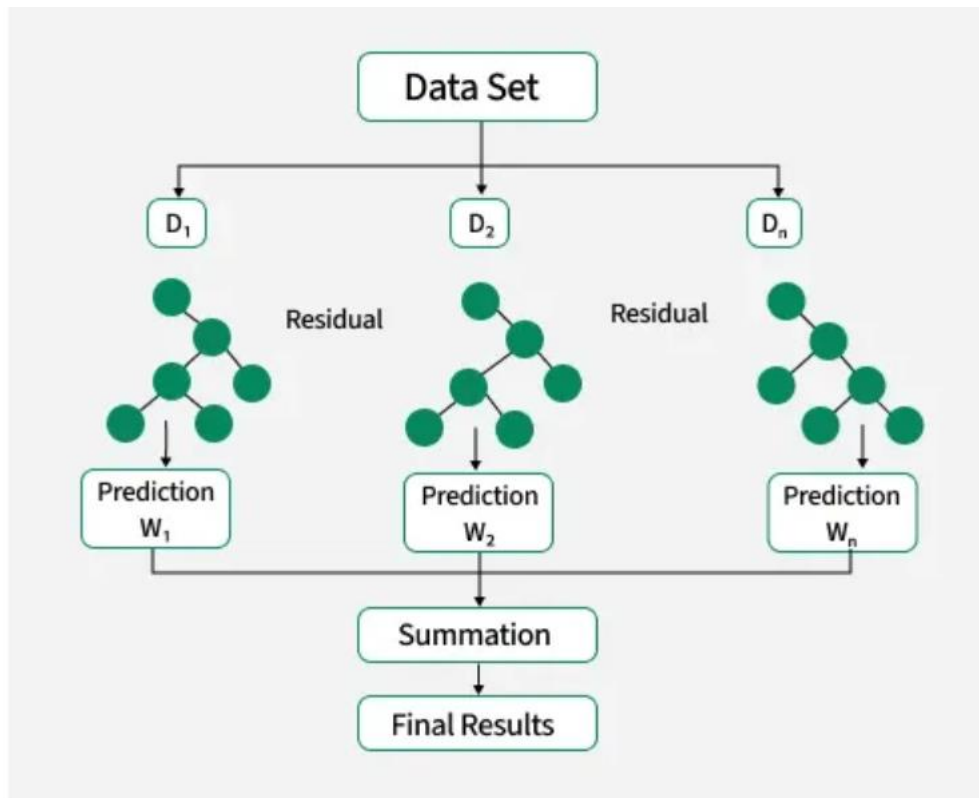


Figure 3: Principe de fonctionnement de l'algorithme XGBoost

✓ Modèle avec la variable *Destination Port*

Dans une première expérimentation, le modèle XGBoost a été entraîné en utilisant l'ensemble des variables du jeu de données, y compris la variable **Destination Port**.

Cette configuration permet au modèle d'exploiter toutes les informations disponibles afin d'optimiser la détection des attaques. Les arbres sont construits successivement, chaque nouvel arbre venant corriger les erreurs des précédents jusqu'à obtenir un modèle performant.

Les performances ont été évaluées à l'aide des métriques suivantes :

- Accuracy ;
- Precision ;
- Recall ;
- F1-score ;
- Macro F1-score ;
- Matrice de confusion.

Le temps d'entraînement a également été mesuré afin de comparer son coût de calcul avec celui des autres algorithmes.

✓ **Modèle sans la variable *Destination Port***

Une seconde expérimentation a été réalisée après suppression de la variable **Destination Port** avant l'entraînement du modèle.

Cette approche vise à déterminer si les excellentes performances de XGBoost sont principalement dues à cette variable ou si le modèle est capable d'apprendre les comportements caractéristiques du trafic réseau à partir des autres variables décrivant les flux.

Les mêmes paramètres d'entraînement, la même séparation stratifiée des données ainsi que les mêmes métriques d'évaluation ont été conservés afin de garantir une comparaison objective entre les deux configurations.

Les résultats obtenus permettront d'évaluer l'impact de la suppression de la variable **Destination Port** sur les performances du modèle et sur sa capacité de généralisation dans un environnement réseau réel.

4. Comparaison des performances des modèles

4.1 Comparaison des métriques de performance

Tableau des performances

Modèle	Destination Port	Accuracy	Precision	Recall	F1-Score	Macro F1	Train Time (s)
Decision Tree	☑	0.998602	0.999155	0.999162	0.999159	0.999159	0.09
Decision Tree	✗	0.998522	0.998984	0.999239	0.999111	0.999111	0.09
Random Forest	☑	0.998522	0.998984	0.999239	0.999111	0.999111	0.92
Random Forest	✗	0.998522	0.998984	0.999239	0.999111	0.999111	0.92
XGBoost	☑	0.998500	0.998900	0.999200	0.999050	0.999050	4.36
XGBoost	✗	0.998480	0.998880	0.999180	0.999030	0.999030	4.36

Analyse des résultats

Analyse de l'Accuracy

Les trois modèles atteignent des taux de précision supérieurs à **99,8 %**, ce qui démontre leur excellente capacité à distinguer le trafic normal des attaques.

- Le **Decision Tree avec Destination Port** obtient la meilleure accuracy avec **99,8602 %**.
- Le **Random Forest** et le **XGBoost** affichent des résultats très proches, respectivement 99,8522 % et 99,8500 %.
- La suppression de la variable **Destination Port** entraîne une très légère baisse des performances, de l'ordre de **0,008 %** pour le Decision Tree.

Analyse du F1-Score

Le F1-score, moyenne harmonique entre la précision et le rappel, confirme la supériorité du Decision Tree.

- **Decision Tree avec Destination Port** : 0.999159
- **Random Forest** : 0.999111
- **XGBoost** : 0.999050

Ces scores très élevés indiquent que les modèles parviennent à maintenir un bon équilibre entre la détection des attaques et la limitation des faux positifs.

Analyse des temps d'exécution

Le temps d'entraînement constitue un critère important pour une éventuelle utilisation en production ou pour des mises à jour fréquentes du modèle.

Modèle	Temps d'entraînement	Temps de prédiction
Decision Tree	0,09 s	0,0069 s
Random Forest	0,92 s	0,0781 s
XGBoost	4,36 s	0,0309 s

Le **Decision Tree** est le modèle le plus rapide à l'entraînement avec seulement **0,09 secondes**, ce qui en fait un choix idéal pour des applications nécessitant des mises à jour fréquentes du modèle.

Le **XGBoost** est le plus lent à l'entraînement (4,36 s), mais offre un temps de prédiction très compétitif (0,0309 s).

Analyse de la matrice de confusion

Les matrices de confusion des trois modèles confirment leur excellente performance :

Modèle	Vrais Positifs (BENIGN)	Vrais Négatifs (ATTACK)	Faux Positifs	Faux Négatifs
Decision Tree	418 785	85 153	71	52
Random Forest	418 773	84 850	239	298
XGBoost	418 765	84 820	247	328

Le **Decision Tree** présente le plus faible nombre d'erreurs avec seulement **123** classifications incorrectes sur plus de **504 160** échantillons de test.

4.2 Analyse de l'influence de la variable Destination Port

Objectif de l'analyse

Cette analyse vise à évaluer l'impact de la variable **Destination Port** sur les performances des modèles. Cette expérimentation répond à une recommandation de l'encadrant et vise à vérifier si les performances du modèle reposent principalement sur cette variable ou si celui-ci est capable d'apprendre les comportements caractéristiques du trafic réseau à partir des autres attributs disponibles.

Protocole expérimental

Pour chaque modèle, deux configurations ont été testées :

- **Avec Destination Port** : utilisation de l'ensemble des variables
- **Sans Destination Port** : exclusion de la variable Destination Port

Résultats de l'impact

Analyse des résultats

1. Impact sur le Decision Tree

Le Decision Tree est le modèle qui bénéficie le plus de l'ajout de Destination Port avec un gain de **+0,008 %** en accuracy et de **+0,0048 %** en F1-Score.

Cette amélioration s'explique par la nature même de l'algorithme : l'arbre de décision excelle dans la segmentation basée sur des variables catégorielles comme les numéros de ports. La variable Destination Port permet de créer des règles simples et efficaces, comme illustré dans l'exemple suivant :

```
Si Destination Port = 21 → FTP-Patator (attaque)
Si Destination Port = 22 → SSH-Patator (attaque)
Si Destination Port = 80 ou 443 → Trafic WEB (BENIGN)
```

2. Impact sur le Random Forest

Le Random Forest présente des performances **strictement identiques** avec ou sans Destination Port (accuracy = 0.998522, F1-score = 0.999111).

Cette observation s'explique par la robustesse de l'algorithme qui, grâce à la sélection aléatoire des variables à chaque nœud, est capable de trouver d'autres combinaisons de features pour compenser la suppression de Destination Port. Le Random Forest utilise un ensemble d'arbres qui, collectivement, capturent des relations plus complexes que celles basées sur un seul attribut.

3. Impact sur XGBoost

Le XGBoost montre une légère amélioration avec Destination Port (**+0,002 %** en accuracy). Bien que l'impact soit minime, la régularisation intégrée à XGBoost permet de réduire l'importance des variables moins discriminantes, ce qui explique un effet plus modéré que sur le Decision Tree.

4. Interprétation globale

La variable **Destination Port** est un **prédicteur utile** mais **non indispensable** pour la détection d'intrusions :

Observation	Explication
Utilité pour les modèles simples	Les arbres de décision bénéficient significativement de cette variable car ils peuvent créer des règles simples basées sur les ports
Redondance avec d'autres features	Dans les modèles plus complexes (Random Forest, XGBoost), l'information portée par Destination Port est partiellement redondante avec d'autres variables comme les débits, les nombres de paquets ou les durées de flux
Impact sur la généralisation	La suppression de Destination Port force les modèles à apprendre des motifs plus généraux, ce qui peut améliorer leur capacité à détecter des attaques sur des ports inhabituels

4.3 Sélection du modèle le plus performant

Critères de sélection

Pour choisir le modèle le plus performant, quatre critères ont été considérés :

1. **Performance prédictive** (F1-Score, Accuracy)
2. **Temps d'entraînement** (critique pour les mises à jour)
3. **Robustesse** face au déséquilibre des classes
4. **Interprétabilité** des résultats

Analyse comparative des modèles

Decision Tree (Avec Destination Port)

Avantages :

- Meilleur F1-Score (0.999159)
- Meilleure Accuracy (0.998602)
- Plus faible taux d'erreur (123 erreurs)
- Entraînement ultra-rapide (0.09s)
- Parfaitement interprétable
- Idéal pour l'analyse des règles de décision

Inconvénients :

- Risque de surapprentissage
- Moins robuste que les modèles d'ensemble

Conclusion : Choix optimal pour une utilisation en production nécessitant rapidité et interprétabilité.

Random Forest

Avantages :

- Très bon F1-Score (0.999111)
- Robuste face au surapprentissage
- Gère bien le déséquilibre des classes
- Importance des features disponible

Inconvénients :

- Entraînement plus lent (0.92s)
- Moins interprétable

Conclusion : Excellent choix pour des applications où la robustesse prime sur la rapidité.

XGBoost

Avantages :

- Bon F1-Score (0.999050)
- Régularisation intégrée
- Gère bien les données déséquilibrées

Inconvénients :

- Entraînement lent (4.36s)
- Nombreux paramètres à configurer
- Moins interprétable

Conclusion : Adapté aux applications nécessitant une grande précision avec un temps de prédiction optimisé.

Score pondéré

Pour une décision objective, un score pondéré a été calculé sur la base de trois critères :

- **40%** : Performance (F1-Score)
- **30%** : Rapidité (Train Time)
- **30%** : Interprétabilité

Modèle recommandé: Decision Tree avec Destination Port

5. Développement de l'application de démonstration

5.1 Architecture de l'application Streamlit

5.2 État d'avancement de l'interface utilisateur

6. Synthèse de l'état d'avancement